

Домашнее задание по теме
«Java Collections. Stream API»

Формулировка задания:

1. Реализовать класс Автомобиль. У класса есть поля, свойства и методы.

Поля класса:

- а) Номер автомобиля;
- б) Модель;
- в) Цвет;
- г) Пробег;
- д) Стоимость.

Обратить внимание на переопределение метода toString, на сеттеры и геттеры, модификаторы доступа полей.

2. Проверить работу в классе Main, методе main.

3. Создать объект Java Collections со списком автомобилей.

4. Используя Java Stream API, вывести (можно сделать любые 2 пункта из 4):

- 1) Номера всех автомобилей, имеющих заданный в переменной цвет colorToFind или нулевой пробег mileageToFind.
- 2) Количество уникальных моделей в ценовом диапазоне от n до m тыс.
- 3) Вывести цвет автомобиля с минимальной стоимостью.
- 4) Среднюю стоимость искомой модели modelToFind

<i>Входные данные:</i>
[НОМЕР_АВТОМОБИЛЯ][МОДЕЛЬ][ЦВЕТ][ПРОБЕГ][ЦЕНА] a123me Mercedes White 0 8300000 b873of Volga Black 0 673000 w487mn Lexus Grey 76000 900000 p987hj Volga Red 610 704340 c987ss Toyota White 254000 761000 o983op Toyota Black 698000 740000 p146op BMW White 271000 850000 u893ii Toyota Purple 210900 440000 l097df Toyota Black 108000 780000 y876wd Toyota Black 160000 1000000 Black, 0L 700_000L, 800_000L Toyota Volvo
<i>Выходные данные:</i>
Автомобили в базе: Number Model Color Mileage Cost a123me Mercedes White 0 8300000 b873of Volga Black 0 673000 w487mn Lexus Grey 76000 900000

p987hj Volga Red 610 704340

c987ss Toyota White 254000 761000

o983op Toyota Black 698000 740000

p146op BMW White 271000 850000

u893ii Toyota Purple 210900 440000

l097df Toyota Black 108000 780000

y876wd Toyota Black 160000 1000000

Номера автомобилей по цвету или пробегу: a123me b873of o983op l097df

y876wd

Уникальные автомобили: 4 шт.

Цвет автомобиля с минимальной стоимостью: Purple

Средняя стоимость модели Toyota: 744200,00

Средняя стоимость модели Volvo: 0,00

Дополнительно:

1. Реализовать ввод и вывод программы в файл *.txt.
2. Вынести методы работы с автомобилем в папку repository интерфейс CarsRepository и его реализацию CarsRepositoryImpl.
3. Доработать программу до следующей структуры:

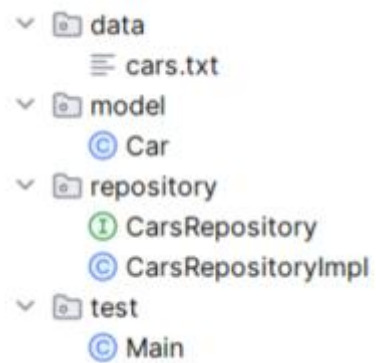


Рисунок 1. Структура проекта

Код программы:

Data/cars.txt:

```
a123me|Mercedes|White|0|8300000  
b873of|Volga|Black|0|673000  
w487mn|Lexus|Grey|76000|900000  
p987hj|Volga|Red|610|704340  
c987ss|Toyota|White|254000|761000  
o983op|Toyota|Black|698000|740000  
p146op|BMW|White|271000|850000  
u893ii|Toyota|Purple|210900|440000  
l097df|Toyota|Black|108000|780000  
y876wd|Toyota|Black|160000|1000000
```

Model/Car.java:

```
package model;  
  
public class Car {  
    private String number;  
    private String model;  
    private String color;  
    private long mileage;  
    private double price;  
  
    public Car(String number, String model, String color, long mileage, double price) {  
        this.number = number;  
        this.model = model;  
        this.color = color;  
        this.mileage = mileage;  
        this.price = price;  
    }  
  
    public String getNumber() {  
        return number;  
    }  
}
```

```
public void setNumber(String number) {
    this.number = number;
}

public String getModel() {
    return model;
}

public void setModel(String model) {
    this.model = model;
}

public String getColor() {
    return color;
}

public void setColor(String color) {
    this.color = color;
}

public long getMileage() {
    return mileage;
}

public void setMileage(long mileage) {
    this.mileage = mileage;
}

public double getPrice() {
    return price;
}

public void setPrice(double price) {
    this.price = price;
}

@Override
public String toString() {
    return number + " " + model + " " + color + " " + mileage + " " + price;
}
}
```

repository/CarsRepository.java:

```
package repository;

import model.Car;
import java.io.IOException;
import java.util.List;

public interface CarsRepository {
    List<Car> getAllCars() throws IOException;
    void saveAllCars(List<Car> cars) throws IOException;
    String getNumbersByColorOrMileage(List<Car> cars);
    long getUniqueCarsCount(List<Car> cars);
    long getUniqueCarsCountauto(List<Car> cars);
    String getMinpriceColor(List<Car> cars);
    double getAverageprice(List<Car> cars, String model);
}
```

repository/CarsRepositoryImpl.java:

```
package repository;

import model.Car;
import java.io.*;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class CarsRepositoryImpl implements CarsRepository {
    private final String ifile = "data/cars.txt";
    private final String ofile = "data/output.txt";

    public CarsRepositoryImpl() throws IOException {
    }

    @Override
    public List<Car> getAllCars() throws IOException {
        List<Car> cars = new ArrayList<>();
    }
}
```

```

File file = new File(ifile);
if (!file.exists()) {
    System.err.println("Файл " + ifile + " не найден.");
    return cars;
}
BufferedReader reader = new BufferedReader(new FileReader(file));
String line;
while ((line = reader.readLine()) != null) {
    String[] parts = line.split("\\|");
    if (parts.length == 5) {
        cars.add(new Car(
            parts[0].trim(),
            parts[1].trim(),
            parts[2].trim(),
            Long.parseLong(parts[3].trim()),
            Double.parseDouble(parts[4].trim())
        ));
    }
}
reader.close();
return cars;
}

@Override
public void saveAllCars(List<Car> cars) throws IOException {
    PrintWriter writer = new PrintWriter(new FileWriter(ofile));
    writer.println("Автомобили в базе:");
    writer.println("Number Model Color Mileage Cost");
    for (Car car : cars) {
        writer.println(car.getNumber() + " " + car.getModel() + " " + car.getColor() + " " + car.getMileage() + " " +
car.getPrice());
    }
    writer.println("Номера автомобилей по цвету или пробегу: " + getNumbersByColorOrMileage(cars));
    writer.println("Уникальные модели: " + getUniqueCarsCount(cars) + " шт.");
    writer.println("Уникальные автомобили: " + getUniqueCarsCountauto(cars) + " шт.");
    writer.println("Цвет автомобиля с минимальной стоимостью: " + getMinpriceColor(cars));
    writer.println("Средняя стоимость модели Toyota: " + getAverageprice(cars, "Toyota"));
    writer.println("Средняя стоимость модели Volvo: " + getAverageprice(cars, "Volvo"));
    System.out.println("\nРезультаты сохранены в: " + ofile);
    writer.close();
}

```



```

@Override
public String getNumbersByColorOrMileage(List<Car> cars) {
    String colorToFind = "Black";
    long mileageToFind = 0L;
    return cars.stream()
        .filter(c -> c.getColor().equalsIgnoreCase(colorToFind) || c.getMileage() == mileageToFind)
        .map(Car::getNumber)
        .collect(Collectors.joining(" "));
}

@Override
public long getUniqueCarsCount(List<Car> cars) {
    long n = 700000;
    long m = 800000;
    return cars.stream()
        .filter(c -> c.getPrice() >= n && c.getPrice() <= m)
        .map(Car::getModel)
        .distinct()
        .count();
}

@Override
public long getUniqueCarsCountauto(List<Car> cars) {
    long n = 700000;
    long m = 800000;
    return cars.stream()
        .filter(c -> c.getPrice() >= n && c.getPrice() <= m)
        .count();
}

@Override
public String getMinpriceColor(List<Car> cars) {
    return cars.stream()
        .min((c1, c2) -> Double.compare(c1.getPrice(), c2.getPrice()))
        .map(Car::getColor)
        .orElse("Не найден");
}

@Override
public double getAverageprice(List<Car> cars, String model) {

```

```

        return cars.stream()
            .filter(c -> c.getModel().equalsIgnoreCase(model))
            .mapToDouble(Car::getPrice)
            .average()
            .orElse(0.0);
    }
}

```

test/Main.java:

```

package test;

import model.Car;
import repository.CarsRepository;
import repository.CarsRepositoryImpl;
import java.util.List;
import java.io.IOException;

public class Main {
    public static void main(String[] args) throws IOException {
        CarsRepository repository = new CarsRepositoryImpl();
        List<Car> cars = repository.getAllCars();

        System.out.println("Автомобили в базе:");
        System.out.println("Number Model Color Mileage Cost");
        for (Car car : cars) {
            System.out.println(car.getNumber() + " " + car.getModel() + " " + car.getColor() + " " + car.getMileage() + " " + car.getPrice());
        }

        System.out.println("Номера автомобилей по цвету или пробегу: " + repository.getNumbersByColorOrMileage(cars));
        System.out.println("Уникальные модели: " + repository.getUniqueCarsCount(cars) + " шт.");
        System.out.println("Уникальные автомобили: " + repository.getUniqueCarsCountauto(cars) + " шт.");
        System.out.println("Цвет автомобиля с минимальной стоимостью: " + repository.getMinpriceColor(cars));
        System.out.println("Средняя стоимость модели Toyota: " + repository.getAverageprice(cars, "Toyota"));
        System.out.println("Средняя стоимость модели Volvo: " + repository.getAverageprice(cars, "Volvo"));

        repository.saveAllCars(cars);
    }
}

```

The screenshot shows an IDE with a project named 'educationjava'. The 'Main.java' file is open, showing the following code:

```
4 import repository.CarsRepository;
5 import repository.CarsRepository;
6 import java.util.List;
7 import java.io.IOException;
8
9 public class Main {
10     public static void main(Stri
11         CarsRepository repositor
12         List<Car> cars = reposit
13
14         System.out.println("Авто
15         System.out.println("Numb
16         for (Car car : cars) {
17             System.out.println(c
```

The 'Run' console shows the output of the program:

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrai
Автомобили в базе:
Number Model Color Mileage Cost
a123me Mercedes White 0 8300000.0
b873of Volga Black 0 673000.0
w487mn Lexus Grey 76000 900000.0
p987hj Volga Red 610 704340.0
c987ss Toyota White 254000 761000.0
o983op Toyota Black 698000 740000.0
p146op BMW White 271000 850000.0
u893ii Toyota Purple 210900 440000.0
l097df Toyota Black 108000 780000.0
y876wd Toyota Black 160000 1000000.0
Номера автомобилей по цвету или пробегу: a123me b873of o983op l097df y876wd
Уникальные модели: 2 шт.
Уникальные автомобили: 4 шт.
Цвет автомобиля с минимальной стоимостью: Purple
Средняя стоимость модели Toyota: 744200.0
Средняя стоимость модели Volvo: 0.0

Результаты сохранены в: data/output.txt

Process finished with exit code 0
```

Скриншот
с выполнением программы
и структура программы: